
Ingeniería de Software 2

Calidad de Software

Calidad de Software

Contenidos:

- Software Quality Assurance.
- Software Quality Management
- Software Process Improvement

Calidad de Software

- Objetivo ultimo de la ingeniería de software:
 - Producir software de **Calidad**.
- La calidad engloba todo el proceso, y esta determinada por factores directos e indirectos.
- La calidad es un concepto complejo y multifacético, que puede describirse desde diferentes perspectivas.

Diferentes Perspectivas



Diferentes Necesidades

Diferentes Sistemas

o

Diferentes partes
de un sistema

Visiones de Calidad

- Visión trascendental
 - Puede ser reconocida pero no definida
- Visión del usuario
 - Grado de adecuación al propósito
- Visión del productor
 - conformidad con la especificación
- Visión del producto
 - ligada a características inherentes del mismo
- Visión basada en valor
 - cuanto esta dispuesto a pagar el cliente?

Visión trascendental

- Ideal Filosófico de calidad.
- La calidad se percibe, no siempre se define.
 - “Este vino es de excelente calidad”, aunque no se detallan métricas.
- Se reconoce cuando está presente.
 - Un concierto con gran acústica y ejecución impecable.
- Basada en la experiencia y la excelencia.
 - La sensación de usar un auto de lujo.

Visión del usuario

- Calidad = satisfacción del cliente.
 - Una app móvil intuitiva y fácil de usar.
- Depende de la adecuación al propósito.
 - Un sistema bancario que permite transferencias sin errores.
- Responde a las necesidades reales del usuario.
 - Un horno con funciones simples, ideal para un hogar básico.

Visión del productor

- Enfoque en cumplir especificaciones.
 - Un módulo desarrollado de acuerdo al documento de requisitos.
- El producto es de calidad si sigue el plan.
 - Un API entregado con todos los endpoints definidos en el diseño.
- Importa la exactitud en el proceso.
 - Software construido con pruebas unitarias que garantizan el código esperado.

Visión del producto

- Calidad ligada a características inherentes.
 - Un sistema con baja latencia y alta disponibilidad.
- Medible: desempeño, confiabilidad, durabilidad.
 - Una plataforma web capaz de soportar 10,000 usuarios concurrentes.
- Objetiva y técnica.
 - Aplicación con 99.9% de uptime documentado.

Visión basada en valor

- Relaciona calidad con el costo y el precio.
 - Un software SaaS con buena relación costo-funcionalidades.
- ¿Cuánto está dispuesto a pagar el cliente?
 - Un CRM accesible frente a uno premium con más integraciones.
- Un producto es de calidad si ofrece valor igual o superior.
 - Una suscripción que incluye soporte técnico 24/7 al mismo precio del mercado.

Software Quality Assurance.

“Una guía planificada y sistemática de todas las acciones necesarias para proveer la evidencia adecuada de que un producto cumple los requerimientos técnicos establecidos.

Un conjunto de actividades diseñadas para evaluar el proceso por el cual un producto es desarrollado o construido.”

Diferentes Roles

- “Como policía del proceso”
- “Como abogado del cliente”
- “Como analista”
- “Como proveedor de información”
- “Como responsable de la elaboración del proceso”

Como policía del proceso

- El trabajo del equipo de SQA es asegurar que el desarrollo sigue el proceso establecido.
 - Auditar los productos del trabajo para identificar deficiencias.
 - Juzgar el proceso y no el producto.
 - Determinar el cumplimiento del plan de desarrollo del proyecto y del proceso de desarrollo de software.

Como abogado del cliente

- El trabajo del equipo de SQA es representar al cliente.
 - Identificar funcionalidades importantes para el cliente.
 - Ayudar el equipo a sensibilizarse con las necesidades del cliente.
 - Actuar como un cliente de prueba.

Como analista

- El trabajo del equipo de SQA es recabar información.
 - Juntar muchos datos sobre todos los aspectos del producto y del proceso.
 - Con esta información ayudar a mejorar los procesos y los productos.

Como proveedor de información

- Revisar qué es lo que esté hecho y decir cuáles objetivos técnicos realmente están cumplidos para que la gerencia pueda tomar mejores decisiones de negocios.
 - Entrega datos técnicos objetivos a la gerencia.
 - Informa sobre tipos de productos y riesgos.
 - Prioriza la reducción de riesgos sobre el cumplimiento estricto del proceso.

Como responsable de la elaboración del proceso

- Participa en la definición de planes, estándares y procedimientos.
- Asegura que los procesos se ajusten a las necesidades del proyecto.
- La calidad debe comenzar desde las fases iniciales.

Software Quality Management

- Es la disciplina que garantiza que el software sea confiable, usable, seguro, mantenible y cumpla los requisitos.
- Se basa en la prevención de errores y en la mejora continua.

Software Quality Management

- **Componentes clave:**

- Planificación de la calidad
 - Definir criterios de aceptación y métricas.
 - Selección de estándares y herramientas.
- Aseguramiento de la calidad (QA)
 - Prevención de errores mediante procesos y auditorías.
 - Revisiones de código, CI/CD, buenas prácticas.
- Control de la calidad (QC)
 - Detección de defectos con pruebas y métricas.
 - Validación y verificación del software.
- Métricas y mejora continua.

Planificación de la Calidad

Definir objetivos y criterios de calidad del producto y del proceso.

- Actividades principales:
 - Identificación de necesidades del cliente.
 - Selección de normas y estándares.
 - Definición de roles, responsabilidades, y recursos.
 - Diseño del plan de calidad detallado.
- Documentos clave:
 - Plan de Gestión de Calidad.
 - Matriz de trazabilidad de requisitos.
 - Lista de criterios de aceptación.

Aseguramiento de la Calidad

Conjunto de procesos sistemáticos para garantizar que tanto las actividades de desarrollo como los productos intermedios cumplan los estándares de calidad establecidos.

- Funciones clave:
 - Auditorías de procesos.
 - Revisión y validación de artefactos.
 - Definición y seguimiento de estándares metodológicos.
- Ventajas:
 - Prevención de defectos.
 - Mayor confianza en el producto final.
 - Reducción de costos de corrección.

Control de Calidad

Actividades orientadas a la revisión, inspección y prueba del software desarrollado para detectar defectos.

- Principales técnicas:
 - Pruebas unitarias, de integración, de sistema y de aceptación.
 - Revisiones de código y auditorías técnicas.
- Indicadores de éxito:
 - Tasa de defectos detectados.
 - Tiempo de resolución de incidencias.
 - Nivel de satisfacción del cliente.

Métricas de Calidad

Medir, cuantificar y analizar características del software y del proceso de desarrollo.

- Tipos de métricas:
 - **Métricas de producto:** defectos, cobertura de pruebas, complejidad, rendimiento.
 - **Métricas de proceso:** productividad, tiempo de entrega, cumplimiento de estándares.
- Indicadores clave:
 - Densidad de defectos.
 - Tasa de rechazo de cambios.
 - Porcentaje de cobertura de pruebas.

Mejora Continua

Proceso cíclico orientado a incrementar de forma sistemática la calidad del producto y del proceso.

- Modelos y metodologías:
 - Ciclo PDCA (Plan-Do-Check-Act).
 - Kaizen y Lean Six Sigma.
 - Lecciones aprendidas y retroalimentación constante.
- Beneficios:
 - Adaptabilidad a cambios del entorno.
 - Mayor competitividad e innovación.
 - Reducción progresiva de errores y desperdicios.

Proceso típico de SQM

- Definir plan de calidad (roles, estándares, herramientas).
- Establecer métricas de calidad (técnicas, de proceso, de negocio).
- Implementar QA (prevención de errores).
- Ejecutar QC (pruebas y mediciones).
- Mejorar procesos de forma continua (CMMI, ISO 25010, retrospectivas).

Normas y Estándares de Calidad

- Normas internacionales:
 - ISO 9001: Gestión de calidad.
 - ISO/IEC 25010: Calidad del producto software.
 - ISO/IEC 12207: Procesos del ciclo de vida del software.
- Aplicación:
 - Definen requisitos mínimos y buenas prácticas.
 - Sirven como referencia para auditorías y certificaciones.
- Importancia:
 - Facilitan la comparabilidad y la confianza.
 - Favorecen la estandarización de procesos.

Modelos de Madurez de Procesos

- Principales modelos:
 - CMMI (Capability Maturity Model Integration).
 - TMMi (Test Maturity Model Integration).
- Niveles de madurez:
 - Nivel 1: Inicial.
 - Nivel 2: Gestionado.
 - Nivel 3: Definido.
 - Nivel 4: Gestionado cuantitativamente.
 - Nivel 5: Optimización continua.
- Aplicaciones y beneficios:
 - Diagnóstico de procesos actuales.
 - Identificación de áreas de mejora.
 - Incremento progresivo de la madurez organizacional.

Herramientas y Automatización

- Tipos de herramientas:
 - Gestión de requisitos (Jira, Trello, Azure Boards).
 - Automatización de pruebas (Selenium, JUnit, TestComplete).
 - Integración continua (Jenkins, GitHub Actions).
 - Análisis de calidad de código (SonarQube, CodeClimate).
- Ventajas de la automatización:
 - Ahorro de tiempo y recursos.
 - Reducción de errores humanos.
 - Mejora en la cobertura y precisión de pruebas.
- Criterios de selección:
 - Compatibilidad con lenguajes y plataformas.
 - Facilidad de integración y mantenimiento.
 - Escalabilidad y soporte técnico.

Roles y Responsabilidades en SQM

- Roles principales:
 - Gerente de calidad: define estrategias y coordina recursos.
 - Analista QA: diseña y ejecuta pruebas.
 - Desarrollador: implementa requisitos de calidad en el código.
 - Product Owner: define y valida criterios de calidad.
- Responsabilidades clave:
 - Comunicación y coordinación entre equipos.
 - Promoción de la cultura de calidad.
 - Implementación y revisión de procesos.
 - Evaluación continua del cumplimiento de estándares.

Proceso típico de SQM

- Definir plan de calidad (roles, estándares, herramientas).
- Establecer métricas de calidad (técnicas, de proceso, de negocio).
- Implementar QA (prevención de errores).
- Ejecutar QC (pruebas y mediciones).
- Mejorar procesos de forma continua (CMMI, ISO 25010, retrospectivas).

Mejores Prácticas

- Buenas prácticas:
 - Definir criterios de calidad claros y mensurables.
 - Integrar QA y QC en todo el ciclo de desarrollo.
 - Automatizar pruebas y registros.
 - Fomentar la retroalimentación y aprendizaje constante.
 - Realizar revisiones y auditorías frecuentes.
- Factores críticos de éxito:
 - Liderazgo comprometido con la calidad.
 - Participación activa del cliente.
 - Adaptación y mejora de procesos según resultados y métricas.
 - Formación continua del personal.

Software Process Improvement

- Conjunto de actividades sistemáticas y estructuradas para analizar, rediseñar, implementar y optimizar los procesos involucrados en el desarrollo y mantenimiento de software.
- Busca aumentar la calidad, productividad, previsibilidad y satisfacción del cliente mediante la estandarización y optimización de procesos técnicos y de gestión.

Importancia

- Permite a las organizaciones adaptarse rápidamente a cambios en el entorno y tecnología.
- Mejora la competitividad y eficiencia operativa en organizaciones orientadas al desarrollo de software.

Objetivos

- Aumentar la calidad del producto y los servicios de software.
- Optimizar los costos y recursos involucrados en los proyectos.
- Mejorar la eficiencia y productividad del equipo de desarrollo.
- Predecir resultados y mantener la conformidad con estándares y regulaciones.
- Satisfacer y superar las expectativas del cliente final.
- Fomentar la innovación mediante la revisión y perfeccionamiento continuo de procesos.

Metas específicas

- Reducir defectos y tiempos de entrega.
- Minimizar los riesgos e incertidumbres en el desarrollo.
- Facilitar la gestión del cambio y la evolución organizacional.

Metodología CMMI - Capability Maturity Model Integration

- Marco de mejores prácticas para la mejora y evaluación de procesos de desarrollo y mantenimiento de software.
- Define cinco niveles de madurez de la organización en términos de gestión de procesos.
- Utilizado globalmente en empresas de tecnología, consultoría y desarrollo industrial.

Estructura de CMMI

- Niveles de madurez:
 - Nivel 1: Inicial (Procesos impredecibles, reactivos)
 - Nivel 2: Gestionado (Procesos planificados, seguimiento básico)
 - Nivel 3: Definido (Procesos estandarizados y documentados)
 - Nivel 4: Gestionado cuantitativamente (Procesos controlados con métricas)
 - Nivel 5: Optimizado (Mejora continua e innovación)

Medición de Calidad

- **Las visiones de calidad afectan las definiciones**
- **Definiciones afectan a las mediciones.**

Visión del Usuario

- Qué valora el usuario
 - Facilidad de uso (usabilidad, experiencia de usuario).
 - Fiabilidad y estabilidad del sistema.
 - Desempeño rápido y capacidad de respuesta.
- Indicadores clave desde el punto de vista del usuario.
 - Tasa de error/reportes de problemas.
 - Tiempo medio de resolución de errores detectados por usuarios.
 - Tasas de adopción, retención, recomendación ("Net Promoter Score").

Visión del Productor

- Intereses del productor
 - Cumplimiento de plazos y presupuesto
 - Uso eficiente de recursos
 - Calidad técnica del código y arquitectura
- Enfoque de la visión del productor
 - Foco en procesos óptimos de desarrollo
 - Prevención y detección temprana de defectos
 - Estabilidad y disponibilidad de entornos de integración continua (CI/CD)

Visión del Producto

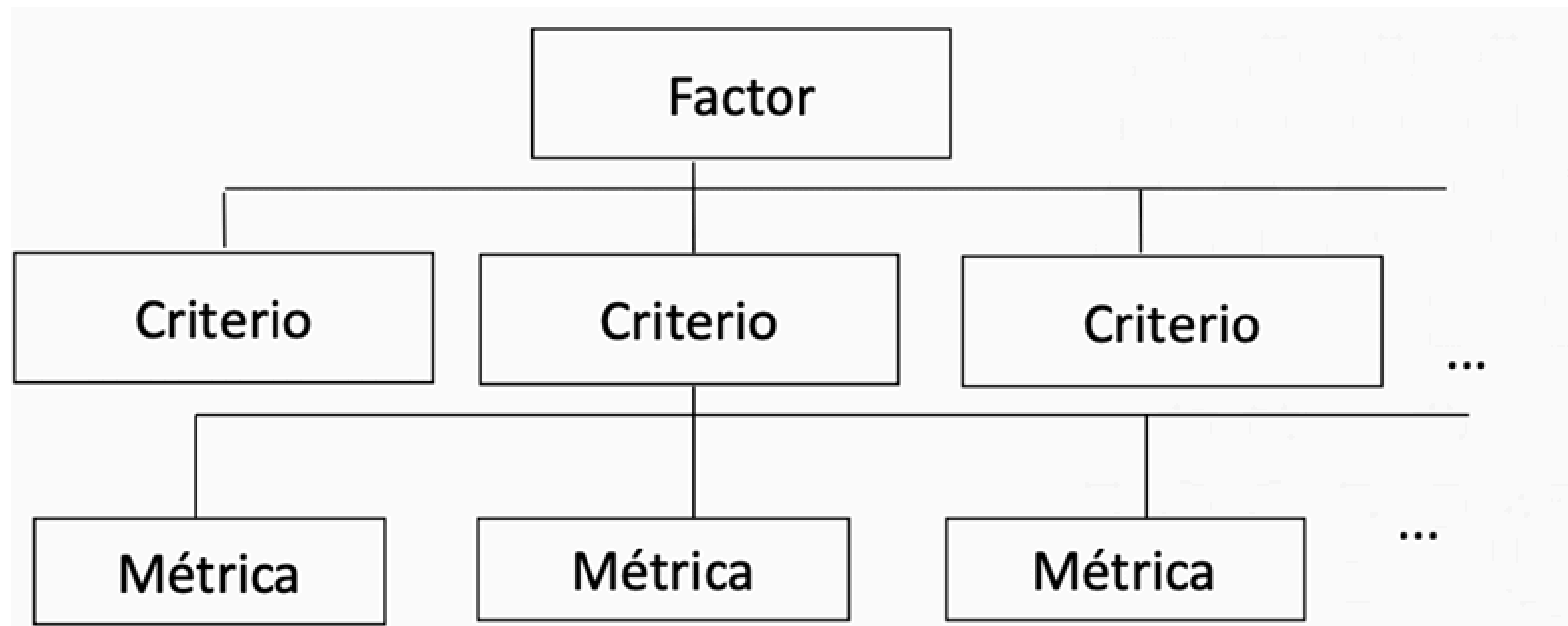
- Foco en las características intrínsecas del software
 - Correctitud funcional
 - Integridad de la arquitectura
 - Documentación adecuada
- Métricas técnicas específicas del producto
 - Número de líneas de código (KLOC)
 - Densidad de defectos por módulo
 - Tamaño y cobertura de documentación técnica
- Atributos claves según estándares
 - Fiabilidad, Mantenibilidad, Eficiencia, Usabilidad, Portabilidad, Seguridad.

Visión Basada en Valor

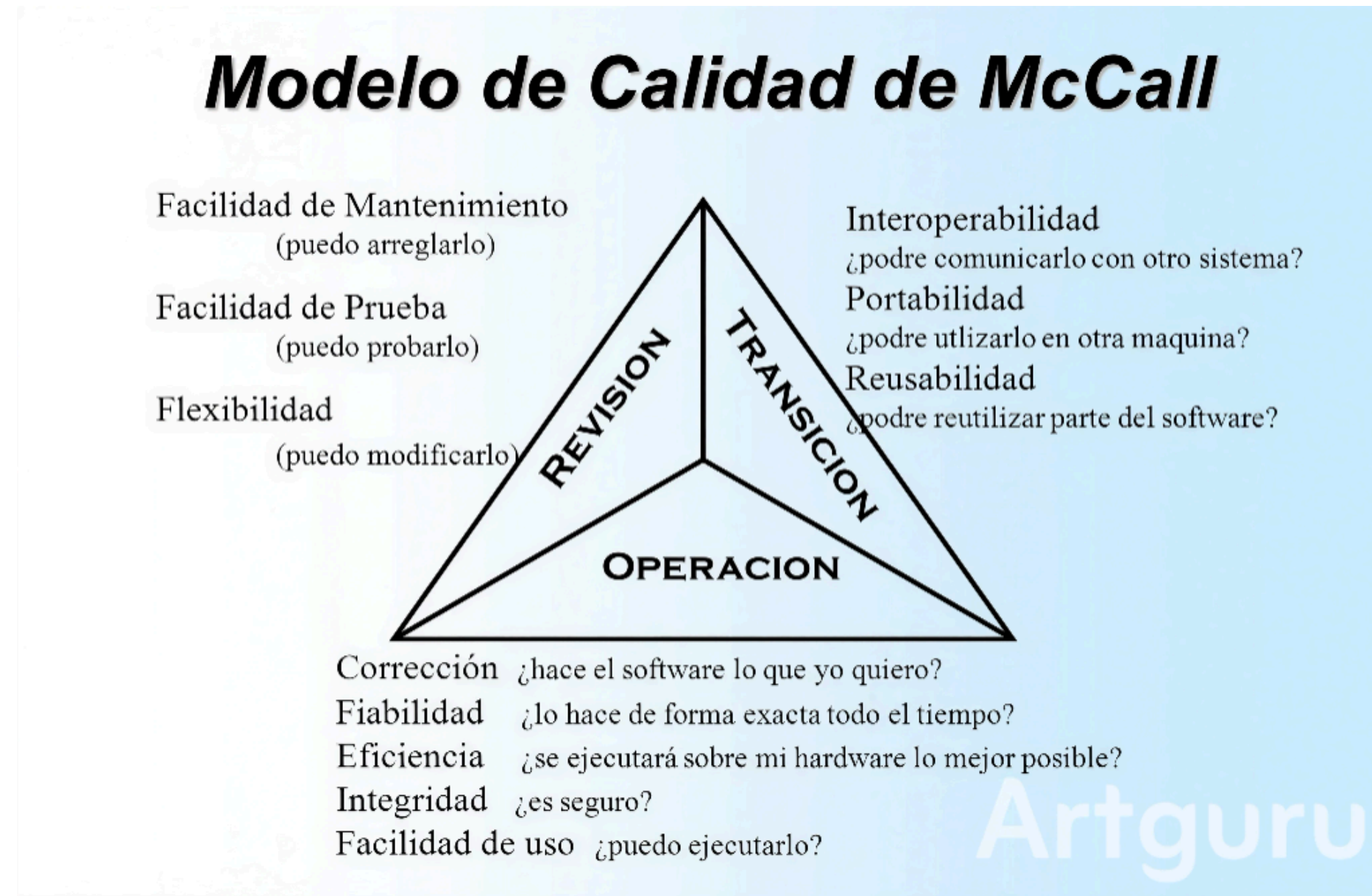
- El valor como criterio central de calidad
 - Medida en que el software contribuye a los objetivos y resultados de negocio
- Criterios usados típicamente Retorno de la inversión (ROI)
 - Reducción de costos operativos
 - Incremento de ingresos o productividad
 - Grado de alineación con la estrategia empresarial
- Indicadores principales de valor
 - Tiempo de comercialización (time to market)
 - Velocidad de adopción por mercados objetivo
 - Satisfacción de stakeholders y alineación con expectativas de negocio

Modelos de Calidad

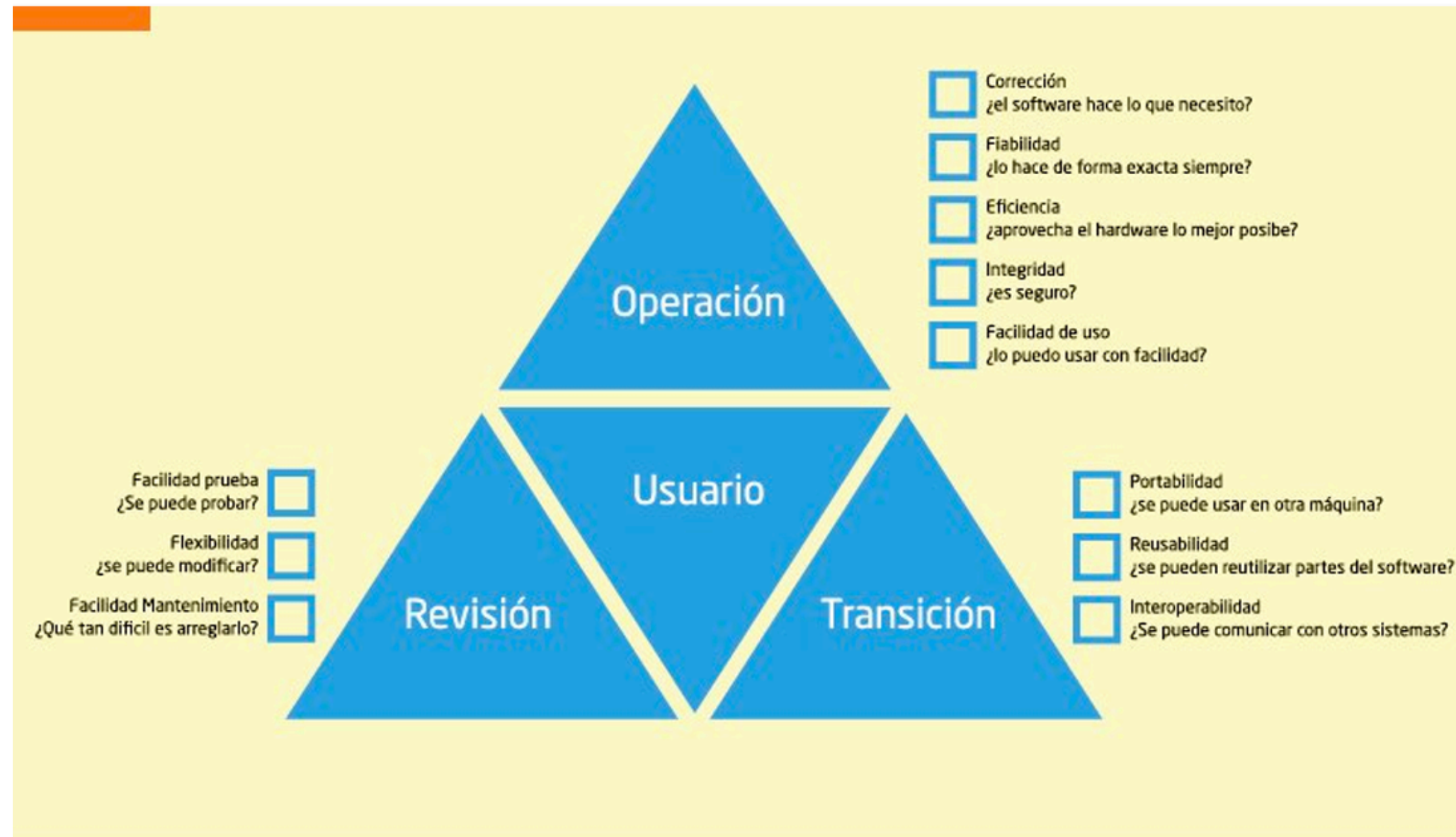
- Buscan reconciliar distintas visiones.
- Ligada a características y factores externos.



Modelo de Calidad de McCall

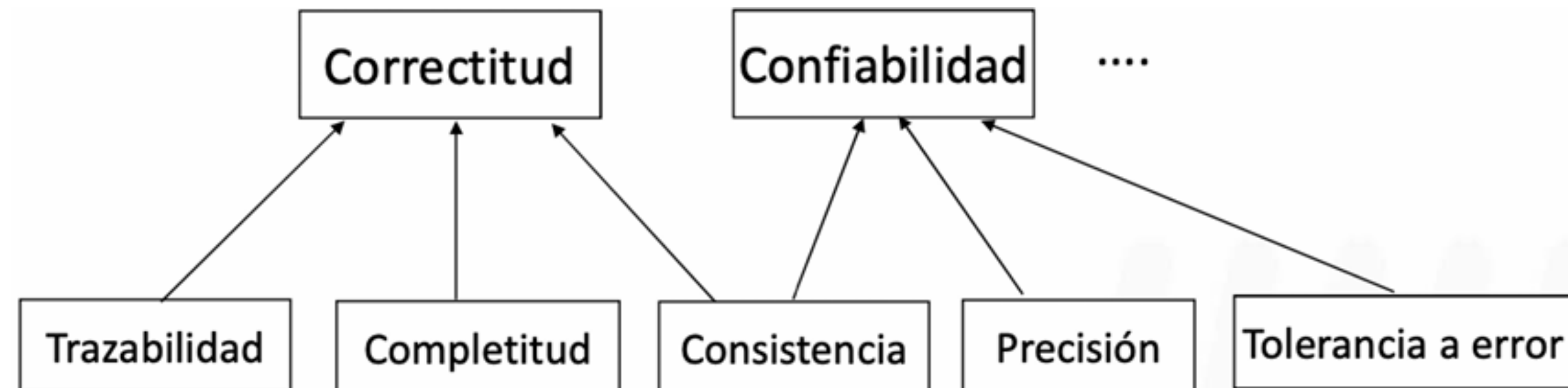


Atributos de Calidad



Factores y Criterios

- Un Criterio puede afectar a mas de un factor.



Norma ISO/IEC 9126

Estándar internacional publicado en 1991 (actualizada en 2001), orientado a la evaluación integral de la calidad del software.

- Objetivos esenciales:
 - Definir un marco uniforme y universal de atributos de calidad.
 - Establecer métricas y procedimientos para la evaluación sistemática.
 - Considerar distintas perspectivas: interna, externa y en uso.
- Estructura jerárquica:
 - 6 características principales de calidad.
 - Se subdividen en subcaracterísticas concretas.
 - Incluye métricas sugeridas para cada aspecto.

Características y Subcaracterísticas

Refleja la
visión del
usuario,
mientras
que McCall
refleja la
visión del
producto

CARACTERÍSTICAS PRINCIPALES	SUBCARACTERÍSTICAS (EJEMPLOS POR CARACTERÍSTICA)
FUNCIONALIDAD	• ADECUACIÓN • EXACTITUD • INTEROPERABILIDAD • SEGURIDAD DE ACCESO • CUMPLIMIENTO FUNCIONAL
FIABILIDAD	• MADUREZ • TOLERANCIA A FALLOS • RECUPERABILIDAD • EXACTITUD DE RESULTADOS
USABILIDAD	• COMPRENSIBILIDAD • FACILIDAD DE APRENDIZAJE • OPERABILIDAD • ATRACTIVIDAD
EFICIENCIA	• COMPORTAMIENTO TEMPORAL • UTILIZACIÓN DE RECURSOS
MANTENIBILIDAD	• FACILIDAD DE ANÁLISIS • FACILIDAD DE CAMBIO • ESTABILIDAD • FACILIDAD DE PRUEBA
PORTABILIDAD	• ADAPTABILIDAD • FACILIDAD DE INSTALACIÓN • COEXISTENCIA • REEMPLAZABILIDAD

Características Internas, Externas y en Uso

- Características internas:
 - Se evalúan mediante el análisis del código fuente, arquitectura y documentación.
 - Ejemplos: Modularidad, legibilidad, complejidad estructural.
- Características externas:
 - Se evalúan mediante la ejecución y pruebas del software en ambientes controlados.
 - Ejemplos: Rendimiento en tiempo real, respuesta ante fallos.
- Calidad en uso:
 - Perspectiva final del usuario en condiciones reales de operación.
 - Dimensiones: Efectividad, productividad, seguridad, satisfacción.

Algunos Problemas

- Difícil de medir: muchas características son cualitativas.
- Algunas subcaracterísticas tienen tanta relevancia que deberían ser características principales.
 - Seguridad de acceso está como subcaracterística de *funcionalidad*, pero muchos expertos consideran que debería ser una característica independiente, dada su importancia transversal.

Otros Modelos de Calidad de Software Relevantes

- Modelo de Boehm
- Modelo FURPS
- Modelo de Dromey

Software Defects

Definición y Diferencias Básicas

- Defecto (Defect/Bug):
 - Anomalía en un producto de software que puede provocar un comportamiento incorrecto o inesperado.
- Error:
 - Acción humana incorrecta que produce un resultado no deseado, frecuentemente causa directa de un defecto.
- Fallo (Failure):
 - Manifestación de un defecto cuando el software falla en cumplir una función esperada.

Tipos Comunes de Defectos

Tipo de Defecto	Descripción breve	Ejemplo
Funcionalidad	No cumple requerimiento funcional	Botón no guarda datos
Rendimiento	Baja velocidad o consumo excesivo	Páginas que cargan lentamente
Interfaz	Fallos en integración o presentación	Datos que no se muestran
Seguridad	Facilita ataques o filtraciones	Inyección de SQL
Usabilidad	Dificulta la experiencia del usuario	Flujos confusos
Compatibilidad	No funciona en ciertos SO o navegadores	Error solo en iOS

Orígenes de Defectos

Origen	Ejemplos típicos	Impacto potencial
Requisitos	Ambigüedad, omisión, contradicción	Funcionalidad incorrecta, insatisfacción de usuario
Diseño	Error arquitectónico, falta de modularidad	Ineficiencia, dificultad de mantenimiento
Codificación	Error lógico, mala gestión de excepciones	Fallos, pérdidas de datos, caídas del sistema
Integración	Fallos de comunicación, incompatibilidad de versiones	Fallas globales, mal funcionamiento de sistemas
Entorno	Faltas de recursos, configuraciones erróneas	Errores no reproducibles, despliegue fallido

Severidades de Defectos

Severidad	Descripcion	Impacto	Ejemplo
Bloqueante	Impide uso total/avanzar, sin solución temporal	Detención completa	Sistema no arranca
Crtica	Daño mayor, corrupción de datos, brechas de seguridad	Pérdida grave, posibles sanciones	Pérdida de registros bancarios
Mayor	Limita funcionalidades primarias, solución parcial	Pérdida productividad, molestias	Reportes erróneos, módulo inoperante
Menor	Afecta solo tareas accesorias, bajo impacto	Molestias menores, sin detención	Botón de ayuda roto
Trivial	Errores cosméticos, sin afectar funcionalidad	Ningún impacto real	Texto mal escrito, ícono mal alineado

Remoción de Defectos

- Fundamental, debe realizarse con frecuencia.
- Ventajas:
 - Detección temprana y coste bajo de corrección.
 - Fomenta la transferencia de conocimiento.
- Desafíos:
 - Requiere tiempo y compromiso del equipo.
 - Puede verse afectada por sesgos internos.

Análisis Estático

Examen del código fuente sin ejecutarlo; automatizable con herramientas especializadas.

Herramienta	Tipo de Lenguaje	Foco principal
SonarQube	Multi-lenguaje	Calidad y seguridad
ESLint	JavaScript	Estilo y errores comunes
Checkmarx	Multi-lenguaje	Vulnerabilidad y seguridad

Pruebas Dinámicas

Ejecución del software para observar su comportamiento ante datos o escenarios definidos

- Técnicas principales:
 - Pruebas de unidad: evalúan módulos individuales.
 - Pruebas de integración: verifican interacciones entre módulos.
 - Pruebas de sistema y aceptación: validan requisitos globales y experiencia del usuario.

Pruebas Automatizadas

- Ventajas:
 - Ejecución repetible y rápida a gran escala.
 - Regresión automatizada tras cambios o despliegues.
 - Reducción de errores humanos en la etapa de validación.

Técnica	Descripción	Ejemplo/Herramienta
Unit testing	Prueba de funciones o métodos individuales	JUnit, NUnit
Integration test	Verificación de interacción entre módulos	TestNG, PyTest
E2E testing	Pruebas de flujo completo del sistema	Cypress, Selenium

Herramientas de Gestión y Seguimiento

Herramienta	Características destacadas	Integraciones
Jira	Workflows avanzados, dashboards	Git, Jenkins
Bugzilla	Open source, personalizable	SCM, emails
Azure DevOps	Integración completa CI/CD	Azure, GitHub

Distribucion de Defectos

Fase del ciclo de vida	% típico de defectos introducidos	% típico de defectos detectados
Requerimientos	40–55%	5–10%
Diseño	20–30%	10–15%
Codificación	10–20%	25–35%
Pruebas (unitarias, integración, sistema)	5–10%	40–55%
Mantenimiento / Producción	<5%	5–10%

Remocion de Defectos

Actividad de Remoción	Defectos Encontrados	Defectos por KLOC	Defectos por Puntos de Función
Revisión de Requerimientos	140	2.8	0.28
Revisión de Diseño	220	4.4	0.44
Inspección de Código	480	9.6	0.96
Test de Integración	160	3.2	0.32
Test de Aceptación	100	2.0	0.20
Total de Defectos	1100	22.0	2.20

Comparación de Eficiencias

Etapa	% Remoción	Defectos Iniciales	Defectos Eliminados	Defectos Restantes
Prototipado	40%	20	8	12
Revisión de Reqs	40%	12	5	7
Revisión de Diseño	15%	7	1	6
Inspección de Código	20%	6	1	5
Test Unitario	1%	5	0	5
Test Funcional	10%	5	1	4
Test del Sistema	10%	4	0	4
Test de "Campo"	20%	4	1	3
TOTAL Eliminados	17			
Eficiencia Total	85%			